HelixQA proxy-bank unblock recipe — helix_proxy

Revision: 1 **Last modified:** 2026-07-01T13:35:00Z

This document persists the **verified** recipe for unblocking a real HelixQA run of the proxy test bank. Every fact below was established during prior investigation and is cited as fact per §11.4.6 — nothing here is re-derived, cloned, or executed. **This is a RECIPE, not an executed action.** Adding submodules is operator territory (§11.4.122): the operator decides which option to take; this document only enumerates the choices and their exact commands.

1. Why the bank is blocked

The runner `tools/helixqa/runner/run_proxy_bank.sh` needs the `helixqa` binary. That binary **will not build in this checkout**: the HelixQA submodule's `submodules/helix_qa/go.mod` carries `replace` directives for 6 own-org sibling modules that are **not vendored here**, so `go build` of the full CLI fails on the unresolved replacements.

The proxy bank itself is complete and honest — it is the missing binary, not the bank, that blocks a real run. Bank: **6 cases** in `tools/helixqa/banks/proxy.yaml`:

- `PRX-HTTP-001` — HTTP forward through Squid
- `PRX-HTTPS-001` — HTTPS (CONNECT) forward through Squid
- `PRX-SOCKS5-001`, `PRX-SOCKS5-002` — Dante SOCKS5
- `PRX-CACHE-001`, `PRX-CACHE-002` — Squid cache miss → hit

2. The 6 replaced modules → vendor path → verified SSH URL

All six are pinned at ref `main`. The vendor path is where a `git submodule add` would place each module for `go.mod` replacement resolution.

Go module	Vendor path	Verified SSH URL (ref `main`)
`digital.vasic.docprocessor`		

Go module	Vendor path	Verified SSH URL (ref `main`)
	`submodules/doc_processor`	`git@github.com:HelixDevelopment/DocProcessor.git`
`digital.vasic.llmorchestrator`	`submodules/llm_orchestrator`	`git@github.com:HelixDevelopment/LLMOrchestrator.git`
`digital.vasic.llmprovider`	`submodules/llm_provider`	`git@github.com:HelixDevelopment/LLMProvider.git`
`digital.vasic.llmsverifier`	`submodules/llms_verifier`	`git@github.com:vasic-digital/LLMsVerifier.git`
`digital.vasic.visionengine`	`submodules/vision_engine`	`git@github.com:HelixDevelopment/VisionEngine.git`
`digital.vasic.security`	`submodules/security`	`git@github.com:vasic-digital/security.git`

3. Three unblock options

Option 1 — Cheapest: point the runner at a prebuilt binary (0 vendoring)

If a prebuilt `helixqa` binary exists anywhere on the host, the runner honors `HELIXQA_BIN` and **skips the build entirely** (verified: `run_proxy_bank.sh` lines 119–120 use `$HELIXQA_BIN` when it is set and executable):

```
HELIXQA_BIN=/path/to/helixqa bash tools/helixqa/runner/
        run_proxy_bank.sh
```

Zero submodules added, zero source touched. This is the recommended first choice whenever a compatible `helixqa` binary is already available.

Option 2 — Full vendor: add all 6 submodules

The full HelixQA CLI build needs **all 6** siblings. Add each at its vendor path, install upstreams per §11.4.36, and initialize recursively:

```
git submodule add git@github.com:HelixDevelopment/
        DocProcessor.git    submodules/doc_processor
git submodule add git@github.com:HelixDevelopment/
        LLMOrchestrator.git  submodules/llm_orchestrator
git submodule add git@github.com:HelixDevelopment/
        LLMProvider.git      submodules/llm_provider
git submodule add git@github.com:vasic-digital/
        LLMsVerifier.git        submodules/llms_verifier
git submodule add git@github.com:HelixDevelopment/
        VisionEngine.git     submodules/vision_engine
```

```
git submodule add git@github.com:vasic-digital/
        security.git            submodules/security

# §11.4.36 — run from each newly-added repo root if it ships
        upstreams/
install_upstreams

git submodule update --init --recursive
```

Operator-territory decision (§11.4.122): this alters the project's submodule set and is not taken autonomously.

Option 3 — Minimal proxy-only build (needs 0 siblings)

Isolate the proxy-relevant executor out of the LLM pipeline so the build no longer pulls in the 6 replaced modules. Move `http_executor.go` into a leaf package, **or** guard it behind a `// go:build httponly` tag that excludes `coordinator.go`, `pipeline.go`, `adapters.go`, and `worker.go` (the files that transitively require the siblings). A proxy-only build needs **0** siblings vendored.

This is an own-org submodule edit — HelixQA is a decoupled, project-not-aware submodule per §11.4.28, so any such change must be made upstream in the submodule, not injected with project-specific context.

4. UNKNOWNs to resolve before cloning (§11.4.6)

- **LLMsVerifier layout risk:** the module root may not be the repo root — it may live under an `llm-verifier/` subdirectory. **Verify the module-root layout before cloning** so the `submodules/llms_verifier` path resolves the `go.mod` replacement correctly.
- **Canonical org for the dual-published modules:** `DocProcessor`, `LLMOrchestrator`, `LLMProvider`, and `VisionEngine` exist under **both** `HelixDevelopment` and `vasic-digital`. The HelixQA `helix-deps.yaml` designates **HelixDevelopment** as canonical for these — use the HelixDevelopment URLs in the table above, not the vasic-digital mirrors.

5. Scope guarantee

Per the task boundary and §11.4.122, this document changes **only** documentation. It does not touch the data plane, source, `config/`, or `tests/`, and it executes none of the commands above. The operator selects and runs the chosen option.